Plano de Implementação do Banco de Dados

DemandaTrack · Projeto Integrador II · PUC-Campinas · Time 2

Preparado por Pedro Tiezo Sales Shimizu · 18/10/2026

Em uma frase
Trocar os dados que hoje ficam na memória do servidor e no navegador por um banco MySQL com 6 tabelas, em 4 semanas, para que o sistema esteja integrado (frontend + API + banco) antes da reunião de 16/11 e da banca.

Situação de hoje

Parte	Como está	Problema
Frontend (5 telas)	Pronto, guarda tudo no localStorage do navegador	Cada computador vê dados diferentes; nada é compartilhado
Backend (API Express)	Estrutura pronta, 2 rotas de 5 feitas, dados em memória	Perde tudo ao reiniciar; 3 arquivos de rota ainda vazios
Banco de dados	Não existe	É critério de nota (NE, item 5) e exigência do Documento de Visão

Por que agora: faltam duas reuniões de orientação (19/10 e 16/11). Se o banco não começar esta semana, a reunião de 16/11 vira construção em vez de ajuste final.

1. Três decisões a confirmar com a equipe e a professora

O banco obriga a escolher um único formato. Hoje front e backend divergem nestes pontos:

Ponto	Front hoje (js/dados.js)	Backend hoje (tipos.ts)	Recomendação
Responsável da demanda	Lista de vários responsáveis	Um único responsável, opcional	**Um único responsável.** É o que o Documento de Visão descreve e evita uma tabela extra.
Tipo da demanda	4 tipos + texto livre ("Personalizado")	Somente os 4 tipos	**Somente os 4 tipos (ENUM).** O documento diz "obrigatoriamente quatro tipos".
Acentos nos valores	"Critica", "Em revisao"	"Crítica", "Em revisão"	**Com acento**, igual ao backend. O front só exibe o que vier da API.

2. O modelo: 6 tabelas

A demanda é o centro do sistema. Tudo o mais existe para dar contexto a ela.

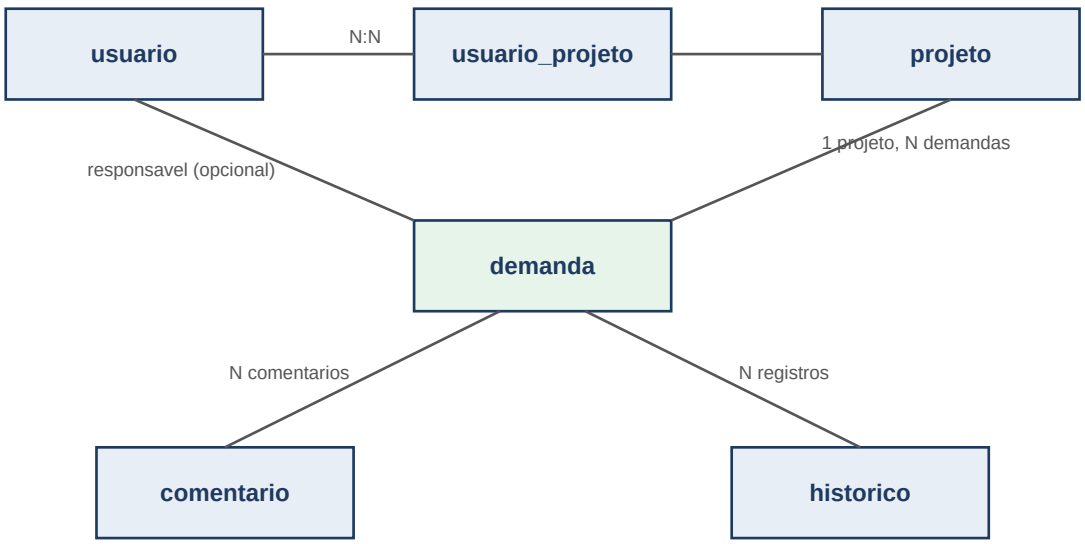

Tabela	O que guarda	Observação
usuario	Nome, e-mail, senha (com hash) e perfil	Perfil é Administrador, Líder de Projeto ou Membro da Equipe
projeto	Nome e descrição	Cadastrado direto no banco (seed), sem tela, como o documento permite
usuario_projeto	Quais usuários pertencem a quais projetos	Líder e Membro só enxergam projetos onde estão. Admin vê tudo
demanda	Título, descrição, tipo, prioridade, status, projeto, responsável, datas e prazo	Status começa em "Aberta" por padrão do banco
comentario	Texto, quem escreveu, em qual demanda e quando	Data e hora preenchidas pelo banco

Tabela	O que guarda	Observação
historico	Frase do que mudou, quem mudou, em qual demanda e quando	Nunca apagado, nem quando a demanda é cancelada

Regras do documento que o próprio banco garante, sem código

- **Status inicial "Aberta"**: valor padrão da coluna.
- **Data de criação e de última atualização automáticas**: CURRENT_TIMESTAMP e ON UPDATE.
- **Responsável pode ficar em branco**: coluna aceita nulo.
- **Sem exclusão física**: o usuário MySQL da aplicação recebe só SELECT, INSERT e UPDATE. Nenhum DELETE passa, nem por erro de programação. Bom argumento para a banca.
- **Valores fixos** de tipo, prioridade, status e perfil: colunas ENUM recusam qualquer outro valor.

O que fica no código do backend: as transições permitidas de status (ex.: não pode ir de Em andamento direto para Concluída), as permissões por perfil e a validação de feriado no prazo.

3. Como o código se conecta ao banco

Princípio: o mínimo de peças novas. Uma dependência nova (o driver) e o resto é recurso nativo do Node.

Peça	Solução	Dependência nova
Driver MySQL	Pacote **mysql2** com pool de conexões e API de promises	Sim (a única)
Configuração (host, usuário, senha)	Arquivo **backend/.env**, lido com process.loadEnvFile() do próprio Node. O .env já está no .gitignore	Não
Conexão	Um arquivo **backend/src/bd.ts** que exporta o pool	Não
Consultas	SQL direto com placeholders "?" dentro de cada arquivo de rota. Sem ORM	Não
Senhas	crypto.scrypt, nativo do Node, para gerar e conferir o hash	Não
Transições de status	Uma função transicaoPermitida(de, para, perfil) no backend	Não

Arquivos novos no repositório

```
database/
  schema.sql      criacao das 6 tabelas (DDL)
  seed.sql        5 usuarios, 3 projetos, 16 demandas e 5 comentarios (os mesmos do dadosTeste.ts)
  usuario.sql     cria o usuario 'app' do MySQL so com SELECT, INSERT, UPDATE
docs/
  mer.png         diagrama entidade-relacionamento
backend/
  .env.example    modelo das variaveis BD_HOST, BD_USUARIO, BD_SENHA, BD_NOME
  src/bd.ts       pool de conexoes
```

Regra do projeto (item 4.5): cada arquivo tem um único autor. Por isso a divisão abaixo é por arquivo.

4. Cronograma: 4 semanas

Semana	Entrega	Quem	Arquivos
1 19 a 25/10	Fundação do banco: instalar MySQL, criar tabelas, carga inicial, conexão no backend, diagrama MER. Front passa a chamar a API.	Pedro	database/*.sql, backend/src/bd.ts, .env.example, docs/mer.png, js/dados.js
2 26/10 a 1/11	Rotas existentes passam a ler do banco; rotas vazias são implementadas.	Nathan: listagem Felipe B.: dashboard e login Pedro: cadastro e detalhes Henrique ou Felipe S.: editar, status e cancelar (PATCH /demandas/:id)	backend/src/rotas/*.ts
3 2 a 8/11	Autorização por perfil (filtra projetos vinculados) e cada tela ligada à API. Sai a barra de simulação de perfil, entra o login real.	Dono de cada tela	js/*.js, pages/*.html, backend middleware
4 9 a 15/11	Testes ponta a ponta dos fluxos do documento. README com instalação do MySQL e execução completa.	Todos; README: Pedro	README.md
16/11	Reunião de orientação com o sistema integrado. Depois: tag 1.0.0-final.	Todos	

5. Passo a passo da semana 1 (Pedro)

#	Passo	Como	Tempo
1	Instalar o MySQL	MySQL Community 8 pelo instalador oficial ou **winget install Oracle.MySQL**. Anotar a senha do root.	30 min
2	Escrever database/schema.sql	Copiar o SQL do anexo deste documento.	30 min
3	Escrever database/seed.sql	Traduzir os dados de backend/src/dados/dadosTeste.ts para INSERTs. Datas relativas viram datas fixas.	1 h
4	Escrever database/usuario.sql	CREATE USER + GRANT sem DELETE.	10 min
5	Rodar e conferir	**mysql -u root -p < database/schema.sql** e depois o seed. Abrir no Workbench e ver as 16 demandas.	20 min
6	Conexão no backend	**npm i mysql2**, criar src/bd.ts com o pool, .env.example. Testar com um SELECT na rota raiz.	1 h
7	Diagrama MER	dbdiagram.io ou MySQL Workbench (reverse engineer), exportar PNG para docs/.	20 min
8	Fechar issues #6, #7, #8, #9	Comentar em cada uma com o arquivo entregue e apontar as horas no GitHub Projects.	15 min
9	js/dados.js chamando a API	Trocar o corpo de lerDemandas, buscarDemanda, salvarDemanda, listarProjetos, listarUsuarios por fetch(). As assinaturas não mudam.	2 h

Para a reunião de 19/10
Levar este documento. Pedir à professora que valide as três decisões da seção 1 (responsável único, tipos fechados, valores com acento). Sair da reunião com as issues #6 a #9 atribuídas a Pedro e a divisão da semana 2 combinada com a equipe.

Riscos e como evitar

Risco	Prevenção
Dois integrantes editando o mesmo arquivo de rota	Divisão por arquivo (seção 4). Cada rota tem um único dono e um único autor no topo.
Front e backend com valores diferentes (acento, responsável)	Decidir na reunião de 19/10 e ajustar o front na semana 1, antes de ligar à API.
MySQL não roda na máquina da banca	README com passo a passo testado em máquina limpa na semana 4. Seed garante dados de demonstração.
Perder histórico ao cancelar demanda	Cancelar é UPDATE de status. Usuário do banco não tem DELETE.

Anexo: SQL de criação (database/schema.sql)

Os valores dos ENUMs seguem exatamente backend/src/tipos.ts, com acento.

```
CREATE DATABASE IF NOT EXISTS demandatrack CHARACTER SET utf8mb4;
USE demandatrack;

CREATE TABLE usuario (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  nome    VARCHAR(100) NOT NULL,
  email   VARCHAR(150) NOT NULL UNIQUE,
  senha   VARCHAR(255) NOT NULL,                -- hash (crypto.scrypt), nunca texto puro
  perfil  ENUM('Administrador','Líder de Projeto','Membro da Equipe') NOT NULL
);

CREATE TABLE projeto (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  descricao TEXT
);

-- vinculo entre usuarios e projetos (Lider e Membro so veem os seus)
CREATE TABLE usuario_projeto (
  usuario_id INT NOT NULL,
  projeto_id INT NOT NULL,
  PRIMARY KEY (usuario_id, projeto_id),
  FOREIGN KEY (usuario_id) REFERENCES usuario(id),
  FOREIGN KEY (projeto_id) REFERENCES projeto(id)
);

CREATE TABLE demanda (
  id               INT AUTO_INCREMENT PRIMARY KEY,
  titulo           VARCHAR(150) NOT NULL,
  descricao        TEXT NOT NULL,
  tipo             ENUM('Tarefa','Defeito','Melhoria','Documentação') NOT NULL,
  prioridade       ENUM('Crítica','Alta','Média','Baixa') NOT NULL,
  status           ENUM('Aberta','Em andamento','Em revisão','Concluída','Cancelada')
                   NOT NULL DEFAULT 'Aberta',
  projeto_id       INT NOT NULL,
  responsavel_id   INT NULL,                    -- pode ficar em branco na criacao
  data_criacao     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  data_atualizacao DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  prazo            DATE NULL,
  FOREIGN KEY (projeto_id)     REFERENCES projeto(id),
  FOREIGN KEY (responsavel_id) REFERENCES usuario(id)
);

CREATE TABLE comentario (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  demanda_id INT NOT NULL,
  usuario_id INT NOT NULL,
  texto      TEXT NOT NULL,
  data_hora  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (demanda_id) REFERENCES demanda(id),
  FOREIGN KEY (usuario_id) REFERENCES usuario(id)
);

CREATE TABLE historico (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  demanda_id INT NOT NULL,
  usuario_id INT NOT NULL,
  descricao  VARCHAR(255) NOT NULL,             -- ex: "alterou o status de Aberta para Em andamento"
  data_hora  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (demanda_id) REFERENCES demanda(id),
  FOREIGN KEY (usuario_id) REFERENCES usuario(id)
);

-- usuario da aplicacao SEM permissao de DELETE: garante "sem exclusao fisica"
CREATE USER IF NOT EXISTS 'app'@'localhost' IDENTIFIED BY 'troque-esta-senha';
GRANT SELECT, INSERT, UPDATE ON demandatrack.* TO 'app'@'localhost';
```